Design Patterns – Grundlagen, Praxis und spannende Klassiker

Einleitung

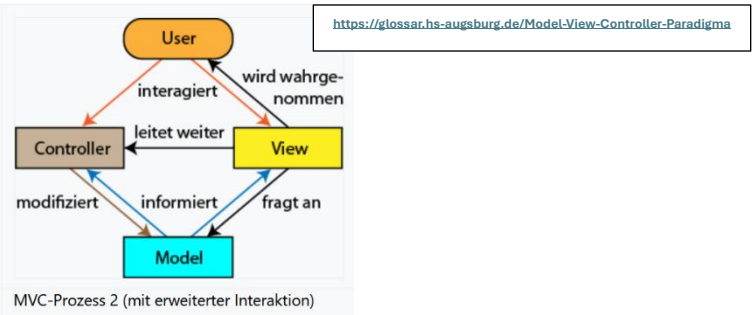
Design Patterns sind bewährte Lösungsansätze zur Strukturierung von Software – egal ob für Apps, Webprojekte oder komplexe IT-Systeme. Sie helfen dabei, Code übersichtlich, wartbar und wiederverwendbar zu gestalten. Besonders interessant sind die folgenden Patterns, die in der Softwareentwicklung immer wieder auftauchen und als "Klassiker" gelten:

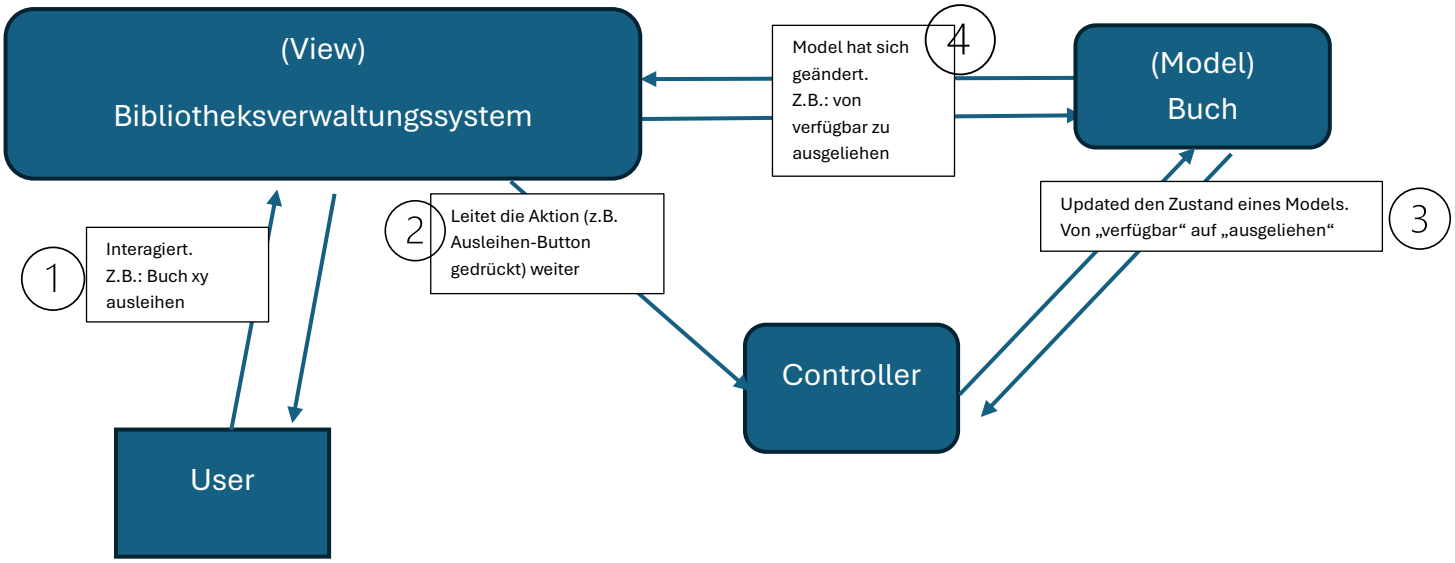
- MVC (Model-View-Controller): Grundpfeiler für viele Apps mit grafischer Oberfläche.
- Singleton: Schlüsseltechnologie für zentrale Ressourcen.
- Factory und Abstract Factory: Die beste Wahl für flexible Objekterstellung.
- Observer: Für automatische Benachrichtigungen und Events in Echtzeit.
- Strategy: Für austauschbare Algorithmen und flexibles Verhalten.
- Command: Ermöglicht Undo/Redo-Funktionen und flexible Steuerung.
- Decorator: Fügt Objekten neue Funktionen hinzu, ohne sie grundlegend zu verändern.
- State und Mediator: Lassen Objekte ihren Zustand oder ihre Kommunikation intelligent steuern.

Diese Muster ermöglichen Anfängern wie Profis die gleichen Vorteile: Klarheit und gute Struktur im eigenen Code.

MVC (Model-View-Controller)

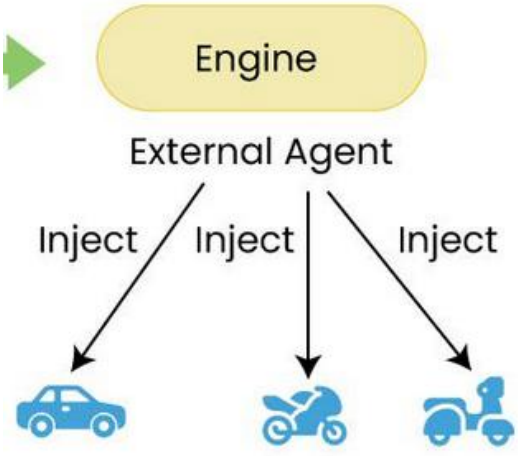
- **Idee:** Trennung von Daten (Model), Darstellung (View) und Logik (Controller).
- **Wird vor allem genutzt, wenn grafische Oberflächen im Spiel sind**
- **Ziel:** Übersichtlichkeit und Wartbarkeit in großen Anwendungen.





Dependency Injection

- **Idee:** Objekte erhalten ihre Abhängigkeiten von außen, statt sie selbst zu erstellen. Somit sollen Klassen eine möglichst geringe Kopplung untereinander haben. Die Folge daraus ist, dass man die Klassen besser einzeln testen kann.



Statt in Auto, Motorrad und Moped immer eigene Motor-Objekte zu erzeugen, werden diese von außen injiziert (Injected). Somit kann man dann auch die Motoren einfacher „auswechseln“.

https://share.google/images/P5kvjBy1RRz9ahfrF

- **Alltägliches Beispiel:** Eine Firma, die spezialisiert ist, für die Wartung von Kaffeemaschinen, bekommt immer Kaffeemaschinen zugeschickt. Es ist egal welche Art von Kaffeemaschine.

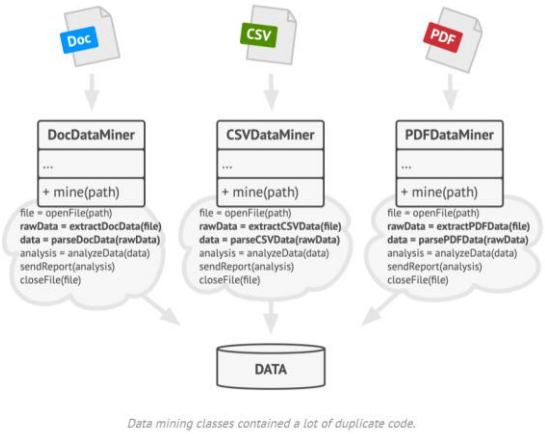
```
class KaffeeAutomat {
    public void brueheKaffee() {
        System.out.println("Kaffee wird gebrüht...");
    }
}

class KaffeeAutomatenWartungsService {
    private KaffeeAutomat automat;

    // Abhängigkeit wird injiziert
    public KaffeeAutomatenWartungsService(KaffeeAutomat automat) {
        this.automat = automat;
    }

    public void fuehreWartungDurch() {
        System.out.println("Wartung gestartet...");
        automat.brueheKaffee();
        System.out.println("Wartung abgeschlossen.");
    }
}

public class Main {
    public static void main(String[] args) {
        KaffeeAutomat automat = new KaffeeAutomat();
        KaffeeAutomatenWartungsService service = new KaffeeAutomatenWartungsService(automat);
        service.fuehreWartungDurch();
    }
}
```

Aufgabe

Implementiere eine Klasse Motor. Injiziere diesen Motor in verschiedene Fahrzeugtypen.

Template Method Pattern

- **Idee:** Legt den Grundablauf eines Algorithmus fest, erlaubt aber Unterklassen, einzelne Schritte zu überschreiben.



https://refactoring.guru/design-patterns/template-method

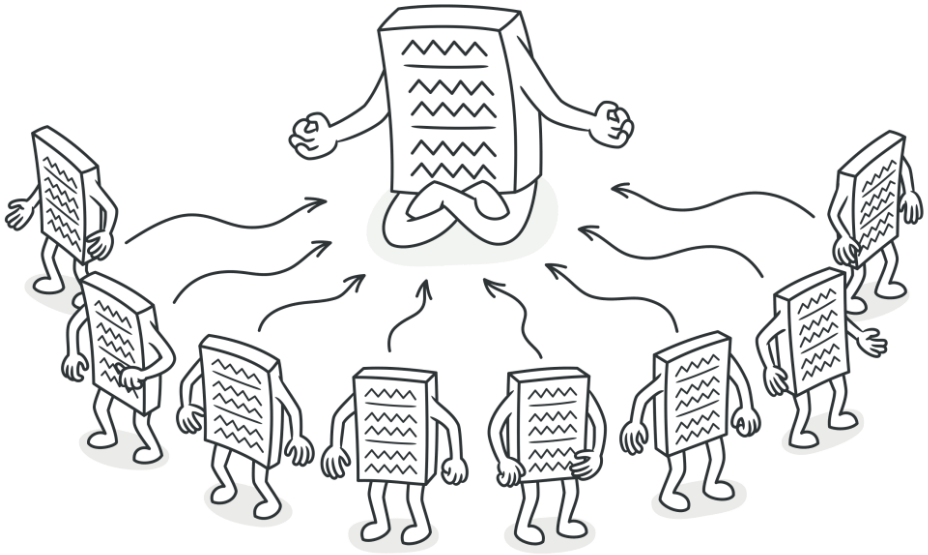
- **Alltägliches Beispiel:** *Zubereitung von Heißgetränken. Immer Wasser aufkochen und immer servieren. Das einzige, was sich unterscheidet ist welche Zutat hinzugefügt wird (Kaffee, Tee, Matcha,...).*
  Anderes Beispiel: Die Auslesung von Dateien erfolgt oft gleich. Datei öffnen, Datei analysieren, Auswertung zurückschicken, Datei schließen. Nur Kleinigkeiten wie: Umwandlung von rohen Docx/Csv/Pdf – Daten in für das System auslesbare Daten unterscheidet sich.

```
abstract class Getraenk {
    public final void zubereiten() {
        wasserKochen();
        zutatHinzufuegen();
        servieren();
    }

    private void wasserKochen() { System.out.println("Wasser kocht..."); }
    protected abstract void zutatHinzufuegen();
    private void servieren() { System.out.println("Getränk wird serviert."); }
}

class Kaffee extends Getraenk {
    protected void zutatHinzufuegen() { System.out.println("Kaffee hinzugefügt."); }
}

class Tee extends Getraenk {
    protected void zutatHinzufuegen() { System.out.println("Tee hinzugefügt."); }
}
```

weiteres Beispiel: https://refactoring.guru/design-patterns/template-method/java/example#lang-features

Singleton

- **Idee:** Nur eine Instanz einer Klasse darf existieren.



- **Ziel:** Gemeinsamer Zugriffspunkt auf zentrale Ressourcen (z. B. Datenbankverbindung).
- **Alltägliches Beispiel:** Ein *Bürgeramt* pro Stadt – alle Anfragen laufen dort zusammen.

```
public class Warenhaus {
    private static Warenhaus instance;

    private Warenhaus() {} // privater Konstruktor

    public static Warenhaus getInstance() {
        if (instance == null) {
            instance = new Warenhaus();
        }
        return instance;
    }

    public void holeLebensmittel(String produkt) {
        System.out.println("Hole " + produkt + " aus dem Warenhaus.");
    }
}

public class Main {
    public static void main(String[] args) {
        Warenhaus wh1 = Warenhaus.getInstance();
        Warenhaus wh2 = Warenhaus.getInstance();
        System.out.println(wh1 == wh2); // true
    }
}
```

Aufgabe

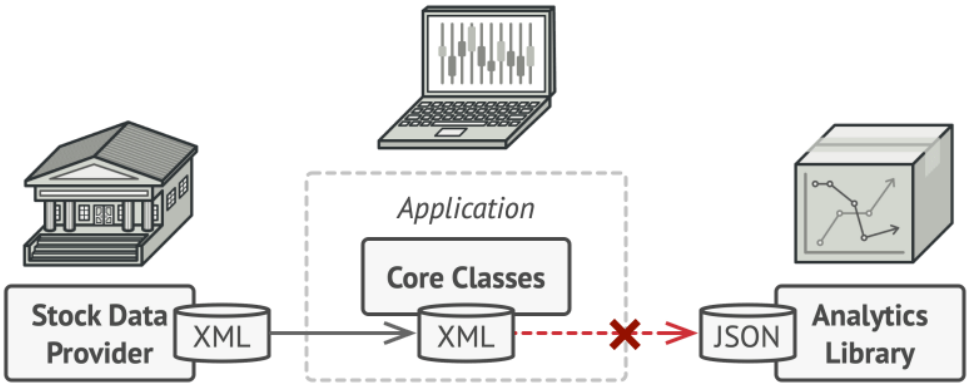
Lese dir folgenden Artikel durch: https://refactoring.guru/design-patterns/singleton

und den dazu gehörigen Code: https://refactoring.guru/design-patterns/singleton/java/example#example-2

Erstelle ein Bürgeramt, auf das Bürger zugreifen können, um ihre Anfragen (Führerschein, Personalausweis beantragen/abholen) zu bearbeiten.

Adapter Pattern

- **Idee:** Erlaubt es Objekten verschiedenen Ursprungs miteinander zu arbeiten.
- **Alltägliches Beispiel:** Ein *Steckdosenadapter* macht einen US-Stecker passend für eine EU-Steckdose.
- **Weiteres Beispiel:**



Es gibt eine Application, welche XML-Dateien bearbeitet. Sie erhält vom Provider XML Dateien. Nun sollen die Daten ausgewertet werden. Der Auswerter akzeptiert nur JSON.

Es muss also eine Schnittstelle her, die XML zu JSON konvertiert.

Lese dir folgenden Artikel durch : https://refactoring.guru/design-patterns/adapter

Nun folgenden Code:

```
// Ziel-Interface - das, was der Client erwartet
interface GermanSocket {
    void plugIn();
}

// Bestehende inkompatible Klasse (z. B. aus einem anderen Land)
class USDevice {
    public void connect() {
        System.out.println("US-Gerät ist mit US-Steckdose verbunden.");
    }
}

// Adapterklasse, die USDevice an GermanSocket anpasst
class USDeviceAdapter implements GermanSocket {
    private USDevice usDevice;

    public USDeviceAdapter(USDevice usDevice) {
        this.usDevice = usDevice;
    }

    @Override
    public void plugIn() {
        System.out.println("Adapter wandelt deutschen Anschluss in US-Format um...");
        usDevice.connect();
    }
}

// Client-Code
public class AdapterExample {
    public static void main(String[] args) {
```

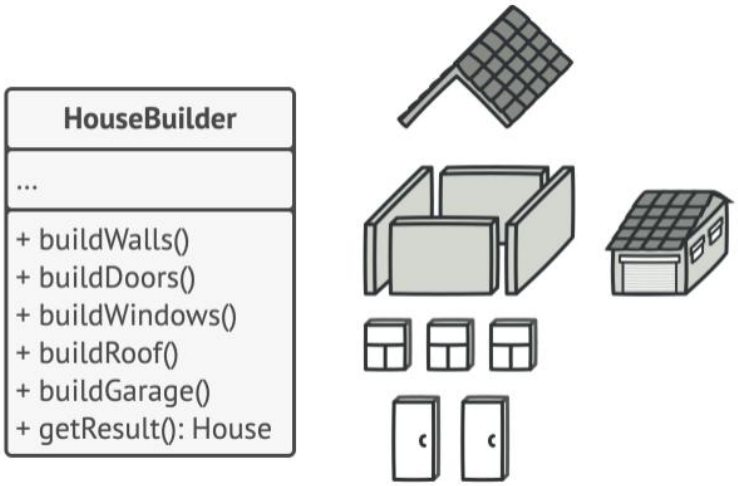
```
        USDevice myUSLamp = new USDevice(); // Gerät mit "falschem" Stecker
        GermanSocket socket = new USDeviceAdapter(myUSLamp); // Adapter verbindet beide Welten

        socket.plugIn(); // Ergebnis: funktioniert über den Adapter
    }
}
```

Aufgabe

Erstelle einen Adapter, der eine alte Druckerklasse (mit printText()) an ein neues Interface ModernPrinter mit printDocument(String text) anpasst.

Builder Pattern

- **Idee:** Baut komplexe Objekte schrittweise auf, anstatt alles über einen langen Konstruktor zu definieren.



https://refactoring.guru/design-patterns/builder

- **Alltägliches Beispiel:** Ein *Burger-Bestellsystem* – man fügt nacheinander Zutaten hinzu.
- **Ohne Builder-**Pattern:

```
public class Burger {
    private String broetchen;
    private String fleisch;
    private boolean kaese;
    private boolean bacon;
    private boolean zwiebeln;
    private boolean salat;
    private boolean tomaten;
    private boolean gurken;
    private String sauce;
    private boolean extraPatty;
    private boolean vegetarisch;
    private boolean vegan;
    private boolean glutenfrei;
    private String groesse; // z. B. "small", "medium", "large"
    private boolean comboMenue; // mit Pommes & Getränk
    private String getraenk;
```

```
    //Langer, schwer zu lesender Konstruktor
    public Burger(String broetchen, String fleisch, boolean kaese, boolean bacon,
                  boolean zwiebeln, boolean salat, boolean tomaten, boolean gurken,
                  String sauce, boolean extraPatty, boolean vegetarisch, boolean vegan,
                  boolean glutenfrei, String groesse, boolean comboMenue, String getraenk) {
        this.broetchen = broetchen;
        this.fleisch = fleisch;
        this.kaese = kaese;
        this.bacon = bacon;
        this.zwiebeln = zwiebeln;
        this.salat = salat;
        this.tomaten = tomaten;
        this.gurken = gurken;
        this.sauce = sauce;
        this.extraPatty = extraPatty;
        this.vegetarisch = vegetarisch;
        this.vegan = vegan;
        this.glutenfrei = glutenfrei;
        this.groesse = groesse;
        this.comboMenue = comboMenue;
        this.getraenk = getraenk;
    }
}
```

Lese dir folgendes durch: https://refactoring.guru/design-patterns/builder

Damit, man den Code sehr strukturiert aufbaut (um die Konzepte der OOP zu beachten), kann der Code schnell lang werden:

```
class Burger {

    private final String bun;
    private final String meat;
    private final boolean cheese;
    private final boolean bacon;
    private final boolean onions;
    private final boolean lettuce;
    private final boolean tomatoes;
    private final boolean pickles;
    private final String sauce;
    private final boolean extraPatty;
    private final boolean vegetarian;
    private final boolean vegan;
    private final boolean glutenFree;
    private final String size;
    private final boolean comboMeal;
    private final String drink;

    Burger(BurgerBuilder builder) {
        this.bun = builder.bun;
        this.meat = builder.meat;
        this.cheese = builder.cheese;
        this.bacon = builder.bacon;
        this.onions = builder.onions;
        this.lettuce = builder.lettuce;
        this.tomatoes = builder.tomatoes;
        this.pickles = builder.pickles;
        this.sauce = builder.sauce;
        this.extraPatty = builder.extraPatty;
        this.vegetarian = builder.vegetarian;
        this.vegan = builder.vegan;
        this.glutenFree = builder.glutenFree;
        this.size = builder.size;
        this.comboMeal = builder.comboMeal;
        this.drink = builder.drink;
    }
}

// The Builder interface defines all setter methods to build components step-by-step
interface Builder{
    Builder setBun(String bun);
    Builder setMeat(String meat);
    Builder setCheese(boolean cheese);
    Builder setBacon(boolean bacon);
    Builder setOnions(boolean onions);
    Builder setLettuce(boolean lettuce);
    Builder setTomatoes(boolean tomatoes);
    Builder setPickles(boolean pickles);
    Builder setSauce(String sauce);
    Builder setExtraPatty(boolean extraPatty);
    Builder setVegetarian(boolean vegetarian);
```

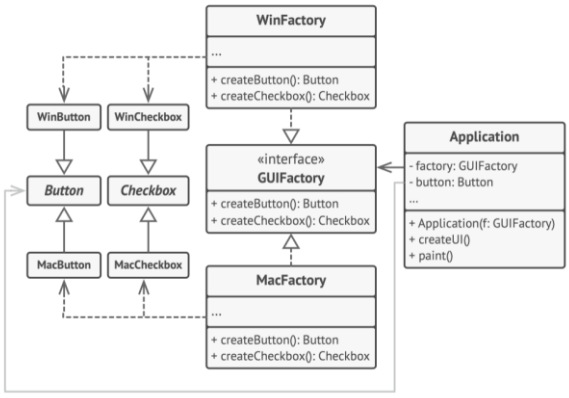
```java
    Builder setVegan(boolean vegan);
    Builder setGlutenFree(boolean glutenFree);
    Builder setSize(String size);
    Builder setComboMeal(boolean comboMeal);
    Builder setDrink(String drink);

    Burger build();
}

// Concrete BurgerBuilder implements the generic Builder interface for Burger
class BurgerBuilder implements Builder {
    String bun;
    String meat;
    boolean cheese;
    boolean bacon;
    boolean onions;
    boolean lettuce;
    boolean tomatoes;
    boolean pickles;
    String sauce;
    boolean extraPatty;
    boolean vegetarian;
    boolean vegan;
    boolean glutenFree;
    String size;
    boolean comboMeal;
    String drink;

    @Override
    public BurgerBuilder setBun(String bun) { this.bun = bun; return this; }
    @Override
    public BurgerBuilder setMeat(String meat) { this.meat = meat; return this; }
    @Override
    public BurgerBuilder setCheese(boolean cheese) { this.cheese = cheese; return this; }
    @Override
    public BurgerBuilder setBacon(boolean bacon) { this.bacon = bacon; return this; }
    @Override
    public BurgerBuilder setOnions(boolean onions) { this.onions = onions; return this; }
    @Override
    public BurgerBuilder setLettuce(boolean lettuce) { this.lettuce = lettuce; return this; }
    @Override
    public BurgerBuilder setTomatoes(boolean tomatoes) { this.tomatoes = tomatoes; return this; }
    @Override
    public BurgerBuilder setPickles(boolean pickles) { this.pickles = pickles; return this; }
    @Override
    public BurgerBuilder setSauce(String sauce) { this.sauce = sauce; return this; }
    @Override
    public BurgerBuilder setExtraPatty(boolean extraPatty) { this.extraPatty = extraPatty; return this; }
    @Override
    public BurgerBuilder setVegetarian(boolean vegetarian) { this.vegetarian = vegetarian; return this; }
    @Override
    public BurgerBuilder setVegan(boolean vegan) { this.vegan = vegan; return this; }
    @Override
    public BurgerBuilder setGlutenFree(boolean glutenFree) { this.glutenFree = glutenFree; return this; }
    @Override
    public BurgerBuilder setSize(String size) { this.size = size; return this; }
    @Override
    public BurgerBuilder setComboMeal(boolean comboMeal) { this.comboMeal = comboMeal; return this; }
    @Override
    public BurgerBuilder setDrink(String drink) { this.drink = drink; return this; }

    @Override
    public Burger build() {
        return new Burger(this);
    }
}

class Main{
    public static void main(String[] args) {
        Burger b = new BurgerBuilder().setBun("Sesame")
                .setMeat("Beef")
                .setCheese(true)
                .setBacon(true)
                .setLettuce(true)
                .setSauce("BBQ")
                .setSize("Large")
                .setComboMeal(true)
                .setDrink("Cola")
                .build();
    }
}
```

Aufgabe

Erstelle ein Menü-Builder. Habe einen Builder mit buildStarter(String s), buildMainCourse(String s), buildDessert(String s), buildDrinks(String s). Nutze dafür aber eine statische Klasse innerhalb der Klasse Menue.

Abstract Factory Pattern

- **Idee:** Eine Klasse (die Factory) erstellt Objekte, ohne dass der Nutzer wissen muss, welche konkrete Klasse instanziiert wird.

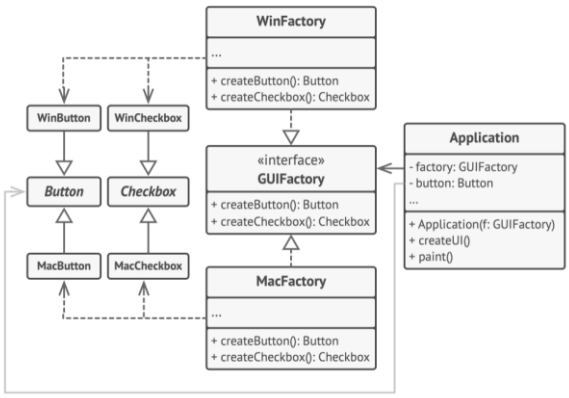


The cross-platform UI classes example.

- **Beispiel:** Die grafische Oberfläche für Windows und Mac sehen unterschiedlich aus. Unser Programm soll je nach Betriebssystem die UI richtig darstellen. Unser Programm überprüft das Betriebssystem und baut dann die benötigte grafische Oberfläche je nach Betriebssystem.

Aufgabe

Schaue dir folgendes Diagramm an:



The cross-platform UI classes example.

https://refactoring.guru/design-patterns/abstract-factory
Schaue dir den dazu gehörigen Code an: https://refactoring.guru/design-patterns/abstract-factory/java/example

Erstelle eine **FahrzeugFactory**, die je nach Distanz ein **Auto**, **Motorrad** oder **Fahrrad** zurückgibt. Bis 30km ein Fahrrad, bis 100km ein Motorrad und ab 100km ein Auto.

Observer Pattern

- **Idee:** Wenn sich ein Objekt ändert, werden automatisch alle abhängigen Objekte informiert.
- **Alltägliches Beispiel:** *Abonnenten einer Zeitung* – wenn eine neue Ausgabe erscheint, bekommen alle Benachrichtigung.
- **Lese dir folgenden Artikel durch:** https://refactoring.guru/design-patterns/observer

```java
import java.util.*;

interface Observer {
    void update(float kurs);
}

class BoersenTicker {
    private List<Observer> beobachter = new ArrayList<>();
    private float aktienkurs;

    public void addObserver(Observer o) { beobachter.add(o); }
    public void removeObserver(Observer o) { beobachter.remove(o); }

    public void setKurs(float kurs) {
        this.aktienkurs = kurs;
        notifyObservers();
    }

    private void notifyObservers() {
        for (Observer o : beobachter) {
            o.update(aktienkurs);
        }
    }
}

class Anleger implements Observer {
    private String name;
    public Anleger(String name) { this.name = name; }
    public void update(float kurs) {
        System.out.println(name + " erhält neuen Kurs: " + kurs);
    }
}

public class Main {
    public static void main(String[] args) {
        BoersenTicker ticker = new BoersenTicker();
        ticker.addObserver(new Anleger("Anna"));
        ticker.addObserver(new Anleger("Ben"));
        ticker.setKurs(123.45f);
    }
}
```

Aufgabe

Baue eine **Wetterstation**, bei der verschiedene Displays (z. B. App, LED-Anzeige -> die Observer) automatisch aktualisiert werden.

Strategy Pattern

https://refactoring.guru/design-patterns/strategy

- **Idee:** Kapselt unterschiedliche Verhaltensweisen oder Algorithmen und macht sie austauschbar.
- **Alltägliches Beispiel:** *Navigations-App*: Wahl zwischen Auto-, Fahrrad- oder Fußweg-Strategien.

```
interface RouteStrategy {
    void route(String start, String ziel);
}

class AutoRoute implements RouteStrategy {
    public void route(String start, String ziel) {
        System.out.println("Auto-Route von " + start + " nach " + ziel);
    }
}

class FahrradRoute implements RouteStrategy {
    public void route(String start, String ziel) {
        System.out.println("Fahrrad-Route von " + start + " nach " + ziel);
    }
}

class Navigation {
    private RouteStrategy strategy;

    public void setStrategy(RouteStrategy strategy) {
        this.strategy = strategy;
    }

    public void navigate(String start, String ziel) {
        strategy.route(start, ziel);
    }
}
```

Aufgabe:

Erstelle Zahlungsmöglichkeiten für eine App

Command Pattern

- **Idee:** Kapselt eine Aktion (Licht einschalten, Datei speichern, E-Mail senden) in einem eigenen Objekt, das als Befehl (Command) bezeichnet wird.
- **Alltägliches Beispiel:** *Ein **Fernseher** kann eingeschaltet, ausgeschaltet oder umgeschaltet werden. Die Fernbedienung soll nur Befehle senden, ohne den Fernseher direkt zu kennen.*

```java
// Das Command-Interface
interface Command {
    void execute();
}

// Der Empfänger, der die eigentliche Aktion ausführt
class Light {
    public void on() { System.out.println("Licht ist an."); }
    public void off() { System.out.println("Licht ist aus."); }
}

// Konkrete Commands
class LightOnCommand implements Command {
    private Light light;
    public LightOnCommand(Light light) { this.light = light; }
    public void execute() { light.on(); }
}

class LightOffCommand implements Command {
    private Light light;
    public LightOffCommand(Light light) { this.light = light; }
    public void execute() { light.off(); }
}

// Der Invoker, der die Befehle ausführt
class RemoteControl {
    private Command command;
    public void setCommand(Command command) { this.command = command; }
    public void pressButton() { command.execute(); }
}

// Anwendung
public class CommandPatternExample {
    public static void main(String[] args) {
        Light livingRoomLight = new Light();
        Command lightOn = new LightOnCommand(livingRoomLight);
        Command lightOff = new LightOffCommand(livingRoomLight);

        RemoteControl remote = new RemoteControl();

        remote.setCommand(lightOn);
        remote.pressButton(); // Licht ist an.

        remote.setCommand(lightOff);
        remote.pressButton(); // Licht ist aus.
    }
}
```

State Pattern

- **Idee:** Verändert das Verhalten eines Objekts, wenn sich sein interner Zustand ändert.
- **Alltägliches Beispiel:** *Ampel*: jede Farbe (Zustand) bestimmt, wie sie sich verhält.

```
interface State { void handle(); }
class Rot implements State { public void handle() { System.out.println("Stopp!"); } }
class Gruen implements State { public void handle() { System.out.println("Fahren!"); } }
class Gelb implements State { public void handle() { System.out.println("Achtung!"); } }

class Ampel {
    private State state;
    public void setState(State s) { this.state = s; }
    public void signal() { state.handle(); }
}
```

Aufgabe: Entwickle ein Musikplayer-State-System (Play, Pause, Stop).

Mediator Pattern

- **Idee:** Vermittelt zwischen Objekten, damit sie nicht direkt miteinander kommunizieren müssen.
- **Alltägliches Beispiel:** *Fluglotse*: koordiniert Flugzeuge, ohne dass sie sich gegenseitig kontaktieren.

```
interface Mediator {
    void send(String message, Kollege sender);
}

abstract class Kollege {
    protected Mediator mediator;
    public Kollege(Mediator m) { this.mediator = m; }
}

class Pilot extends Kollege {
    public Pilot(Mediator m) { super(m); }
    public void senden(String msg) { mediator.send(msg, this); }
    public void empfangen(String msg) { System.out.println("Pilot empfängt: " + msg); }
}

class Tower implements Mediator {
    private List<Kollege> kollegen = new ArrayList<>();
    public void registriere(Kollege k) { kollegen.add(k); }
    public void send(String msg, Kollege sender) {
        for (Kollege k : kollegen) {
            if (k != sender) ((Pilot) k).empfangen(msg);
        }
    }
}
```

Aufgabe:

Simuliere einen Chatroom, in dem Benutzer über einen Mediator Nachrichten senden.

Decorator Pattern

- **Idee:** Fügt Objekten dynamisch neue Funktionen hinzu, ohne deren Klasse zu ändern.
- **Sehr häufig mit java.**io genutzt. FileReader wird in einen BufferedReader gepackt, um Funktionalitäten zu erweitern, ohne die Funktion des FileReaders zu ändern.
- **Alltägliches Beispiel:** *Geschenkverpackung*: Du fügst einem Base-Objekt (dem Geschenk) dekorative Schichten hinzu (Papier, Schleife, Karte).

```java
interface Coffee {
    String getDescription();

    double cost();
}

class SimpleCoffee implements Coffee {
    public String getDescription() {
        return "Kaffee";
    }

    public double cost() {
        return 2.0;
    }
}

abstract class CoffeeDecorator implements Coffee {
    protected Coffee decoratedCoffee;

    public CoffeeDecorator(Coffee coffee) {
        this.decoratedCoffee = coffee;
    }

    public String getDescription() {
        return decoratedCoffee.getDescription();
    }

    public double cost() {
        return decoratedCoffee.cost();
    }
}

class MilkDecorator extends CoffeeDecorator {
    public MilkDecorator(Coffee coffee) {
        super(coffee);
    }

    public String getDescription() {
        return super.getDescription() + ", Milch";
    }

    public double cost() {
        return super.cost() + 0.5;
    }
}
```